

Reserva Inteligente de Restaurantes

Objetivo

El objetivo de esta primera entrega es que los estudiantes desarrollen una API REST para la gestión de reservas en restaurantes, implementando autenticación con JWT, contenedorización con Docker y pruebas unitarias. Esta etapa se centra en la creación de una base funcional y bien estructurada para el sistema completo.

Descripción de la Tarea

Los estudiantes deberán desarrollar una API REST que permita gestionar usuarios, autenticación y reservas de mesas en restaurantes. Se espera que la API sea completamente documentada y contenedorizada con Docker.

Requerimientos y Casos de Uso

1. Autenticación y Gestión de Sesiones

- Implementación de autenticación basada en JWT como un servicio separado.
- Se recomienda utilizar un servicio de autenticación como Keycloak, Aws Cognito o Azure Active Directory; corriendo en su propio contenedor Docker.
- Registro y login de usuarios (clientes y administradores de restaurantes).
- Middleware para proteger rutas y verificar permisos de acceso.

2. Gestión de Usuarios

- Creación, actualización y eliminación de usuarios.
- Diferenciación entre clientes y administradores.

3. Gestión de Restaurantes

- Registro y administración de restaurantes por parte de administradores.
- Consulta de restaurantes disponibles.

4. Gestión de Menús y Productos

- CRUD de platos y precios por restaurante.
- Consulta de menús por restaurante.

5. Reserva de Mesas y Pedidos

- Creación y cancelación de reservas por parte de clientes.
- Verificación de disponibilidad de mesas.
- Realización de pedidos con opción de recogida en el restaurante.

Endpoints Esperados

Método	Endpoint	Descripción
POST	/auth/register	Registro de un nuevo usuario.
POST	/auth/login	Inicio de sesión y obtención de JWT.
GET	/users/me	Obtener detalles del usuario autenticado.
PUT	/users/:id	Actualizar información de un usuario.
DELETE	/users/:id	Eliminar un usuario.
POST	/restaurants	Registrar un restaurante (solo administradores).
GET	/restaurants	Listar restaurantes disponibles.
POST	/menus	Crear un nuevo menú para un restaurante.
GET	/menus/:id	Obtener detalles de un menú específico.
PUT	/menus/:id	Actualizar un menú existente.
DELETE	/menus/:id	Eliminar un menú.
POST	/reservations	Crear una nueva reserva.
DELETE	/reservations/:id	Cancelar una reserva.
POST	/orders	Realizar un pedido.
GET	/orders/:id	Obtener detalles de un pedido.

Cuadro 1
Listado de Endpoints para la API

Contenedorización con Docker

- Crear un Dockerfile para la API, asegurando la instalación de todas las dependencias.
- Utilizar la imagen oficial de PostgreSQL en Docker Hub para la base de datos.
- Contenerizar el servicio de autenticación por separado, si aplica.

Orquestación con Docker Compose

- Definir un archivo `docker-compose.yml` que incluya servicios para la API, la base de datos y el servicio de autenticación.

Pruebas Unitarias y de Integración

- Implementación de pruebas unitarias para cada endpoint clave.
- Desarrollo de pruebas de integración para validar la correcta interacción entre la API y la base de datos.
- Se debe alcanzar un **coverage mínimo del 90%** en las pruebas.

Entregables

- Código Fuente de la API.
- Archivo `Dockerfile`.
- Archivo `docker-compose.yml`.
- Instrucciones de uso en un archivo `README.md`.
- Video demostrativo explicando la implementación.

Evaluación

- **Funcionalidad (50%)**: La API debe cumplir con todos los requerimientos especificados.
- **Contenedorización y Orquestación (30%)**: Uso correcto de Docker y Docker Compose.
- **Documentación (20%)**: Claridad en la documentación y uso de herramientas como Swagger.

Fecha de Entrega y Penalización por Retardo

Los estudiantes deberán subir un archivo comprimido con todos los entregables en el TecDigital antes de las 10:00pm del día de entrega. Si la entrega se realiza después de la hora establecida, se penalizará con una reducción del 5% cada 24 horas.